

Университет ИТМО
Информационные системы

Отчёт по лабораторной работе №1

Управление транспортными средствами
Вариант 84121905

Выполнил: Лейковский Никита
ИСУ 466497

Санкт-Петербург
2026

Содержание

1	Текст задания	3
2	UML-диаграммы	7
3	Исходный код	13
4	Выводы	13

1 Текст задания

Реализовать информационную систему, которая позволяет взаимодействовать с объектами класса `Vehicle`, описание которого приведено ниже.

Модель данных

Класс `Vehicle` содержит следующие закрытые поля. Типы и ограничения соответствуют варианту 84121905.

Поле	Тип	Ограничения
<code>id</code>	<code>Integer</code>	Не null; больше 0; уникальное; генерируется автоматически.
<code>name</code>	<code>String</code>	Не null; строка не может быть пустой.
<code>coordinates</code>	<code>Coordinates</code>	Не null.
<code>creationDate</code>	<code>java.util.Date</code>	Не null; генерируется автоматически.
<code>type</code>	<code>VehicleType</code>	Не null.
<code>enginePower</code>	<code>int</code>	Больше 0.
<code>numberOfWheels</code>	<code>Integer</code>	Не null; больше 0.
<code>capacity</code>	<code>Double</code>	Не null; больше 0.
<code>distanceTravelled</code>	<code>int</code>	Больше 0.
<code>fuelConsumption</code>	<code>double</code>	Больше 0.
<code>fuelType</code>	<code>FuelType</code>	Не null.

Класс `Coordinates` содержит закрытые поля `x` типа `Double` и `y` типа `Float`. Оба поля не могут быть null. Максимальное значение `y`: 408.

Перечисление `VehicleType`: `PLANE`, `BOAT`, `SHIP`. Перечисление `FuelType`: `DIESEL`, `ALCOHOL`, `MANPOWER`.

Основные требования

1. Основное назначение информационной системы - управление объектами, созданными на основе заданного в варианте класса.
2. Необходимо, чтобы с помощью системы можно было выполнить следующие операции с объектами: создание нового объекта, получение информации об объекте по ИД, обновление объекта (модификация его атрибутов), удаление объекта. Операции должны осуществляться в отдельных окнах (интерфейсах) приложения. При получении информации об объекте класса должна также выводиться информация о связанных с ним объ-

ектах.

3. При создании объекта класса необходимо дать пользователю возможность связать новый объект с объектами вспомогательных классов, которые могут быть связаны с созданным объектом и уже есть в системе.
4. Выполнение операций по управлению объектами должно осуществляться на серверной части (не на клиенте), изменения должны синхронизироваться с базой данных.
5. На главном экране системы должен выводиться список текущих объектов в виде таблицы (каждый атрибут объекта - отдельная колонка в таблице). При отображении таблицы должна использоваться пагинация (если все объекты не помещаются на одном экране).
6. Нужно обеспечить возможность фильтровать/сортировать строки таблицы, которые показывают объекты (по значениям любой из строковых колонок). Фильтрация элементов должна производиться только по полному совпадению.
7. Переход к обновлению (модификации) объекта должен быть возможен из таблицы с общим списком объектов и из области с визуализацией объекта (при её реализации).
8. При добавлении/удалении/изменении объекта он должен автоматически появиться/исчезнуть/измениться в интерфейсах у других пользователей, авторизованных в системе.
9. Если при удалении объекта с ним связан другой объект, связанные объекты должны быть связаны с другим объектом (по выбору пользователя), а изначальный объект удалён.
10. Пользователи должны иметь возможность просмотра всех объектов. Для модификации объекта должно открываться отдельное диалоговое окно. При вводе некорректных значений в поля объекта должны появляться информативные сообщения о соответствующих ошибках.

Специальные операции

В системе должен быть реализован отдельный пользовательский интерфейс для выполнения специальных операций над объектами:

1. Удалить все объекты, значение поля `fuelType` которого эквивалентно заданному.
2. Удалить один (любой) объект, значение поля `fuelConsumption` которого

эквивалентно заданному.

3. Вернуть количество объектов, значение поля fuelConsumption которых больше заданного.
4. Найти все транспортные средства заданного типа.
5. Найти все транспортные средства с мощностью двигателя в заданном диапазоне.

Представленные операции должны быть реализованы в рамках компонентов бизнес-логики приложения без прямого использования функций и процедур БД.

Хранение объектов

1. Организовать хранение данных об объектах в реляционной СУБД PostgreSQL. Каждый объект, с которым работает ИС, должен быть сохранён в базе данных.
2. Все требования к полям класса, указанные в виде комментариев к описанию классов, должны быть выполнены на уровне ORM и БД.
3. Для генерации поля id использовать средства базы данных.
4. Для подключения к БД на кафедральном сервере использовать хост pg, имя базы данных - studs, имя пользователя/пароль совпадают с таковыми для подключения к серверу.

Взаимодействие с пользователем

1. Система должна реагировать на некорректный пользовательский ввод, ограничивая ввод недопустимых значений и информируя пользователей о причине ошибки.
2. Переходы между различными логически обособленными частями системы должны осуществляться с помощью меню.
3. При добавлении/удалении/изменении объекта он должен автоматически появиться/исчезнуть/измениться на области у всех других клиентов.

Требования к разработке

1. В качестве основы для реализации ИС необходимо использовать Spring MVC.
2. Для создания уровня хранения необходимо использовать Hibernate.
3. Разные уровни приложения должны быть отделены друг от друга, разные

логические части ИС должны находиться в отдельных компонентах.

Состав отчёта

Текст задания; UML-диаграммы классов и пакетов разработанного приложения; исходный код системы или ссылка на репозиторий с исходным кодом; выводы по работе. Репозиторий при сдаче должен быть публичным; если ветка не указана, используется основная.

2 UML-диаграммы

Классы предметной области

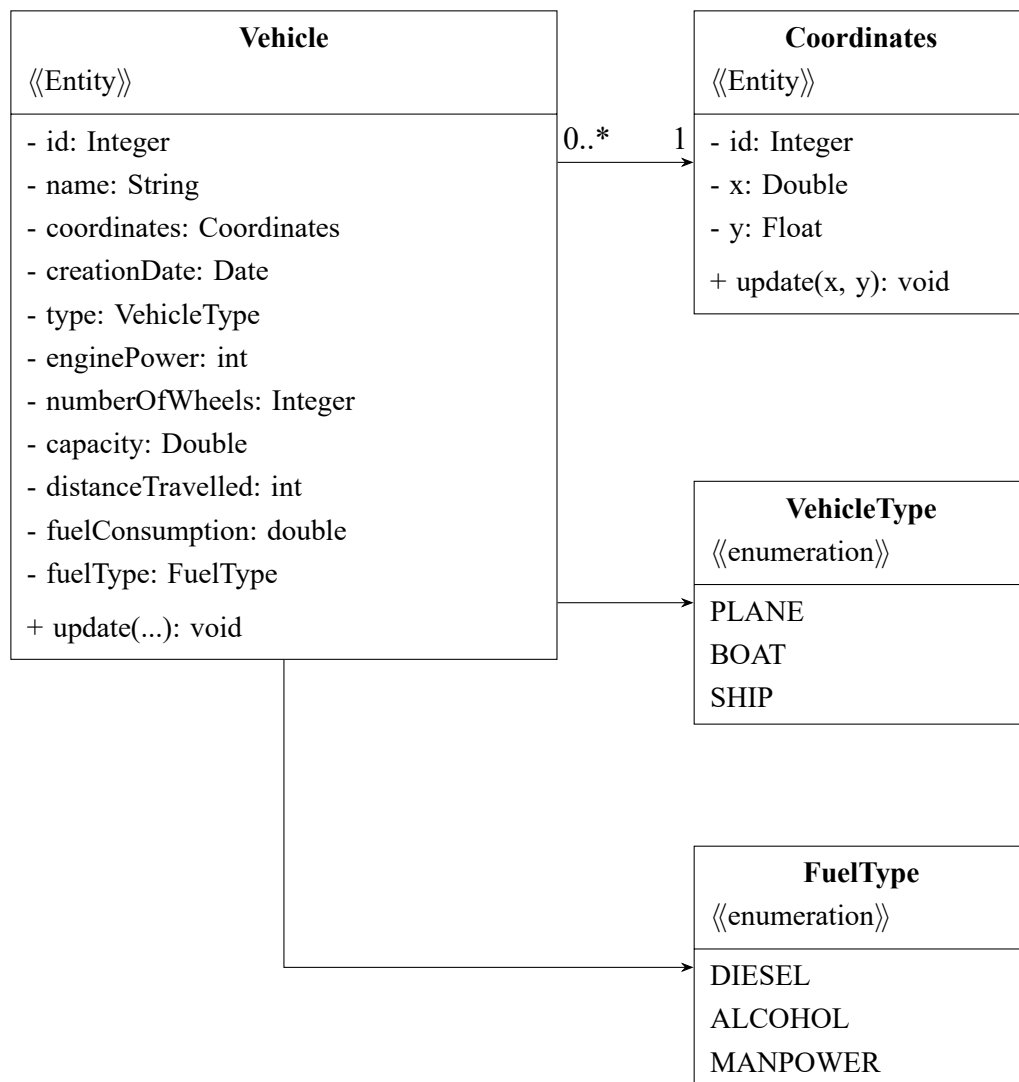


Рисунок 1 - Классы предметной области

Coordinates является отдельной сущностью. Несколько объектов **Vehicle** могут ссылаться на один набор координат через внешний ключ `coordinates_id`. Стрелки обозначают направленные ассоциации. Методы доступа на диаграмме опущены.

Классы приложения

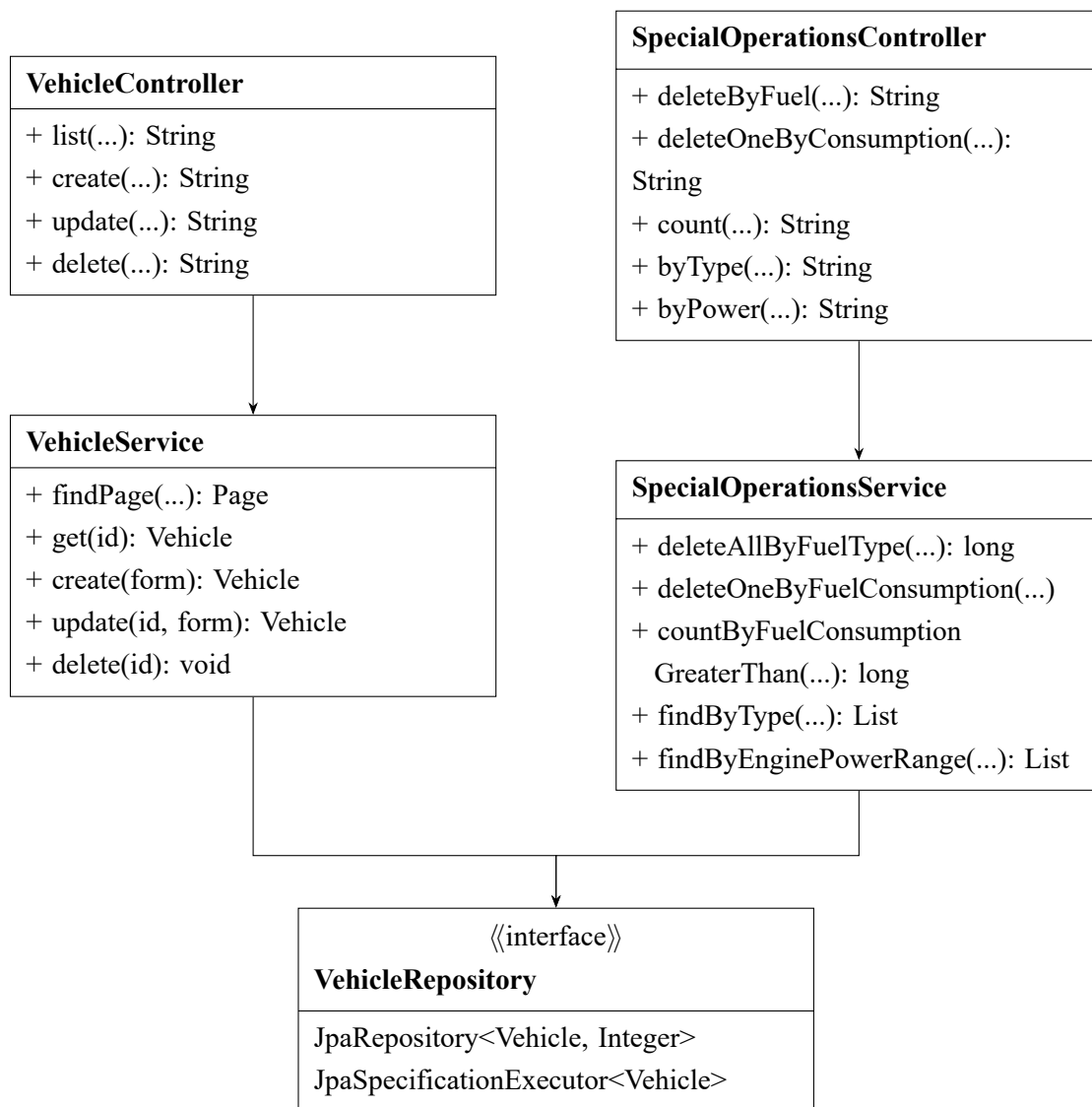


Рисунок 2 - Контроллеры, сервисы и репозиторий

Контроллер `VehicleController` получает данные через DTO `VehicleForm`. Сервис `VehicleService` использует этот DTO при создании и изменении объектов. Параметры методов сокращены для читаемости. `VehicleRepository` расширяет интерфейсы Spring Data JPA.

Связанные объекты и вход

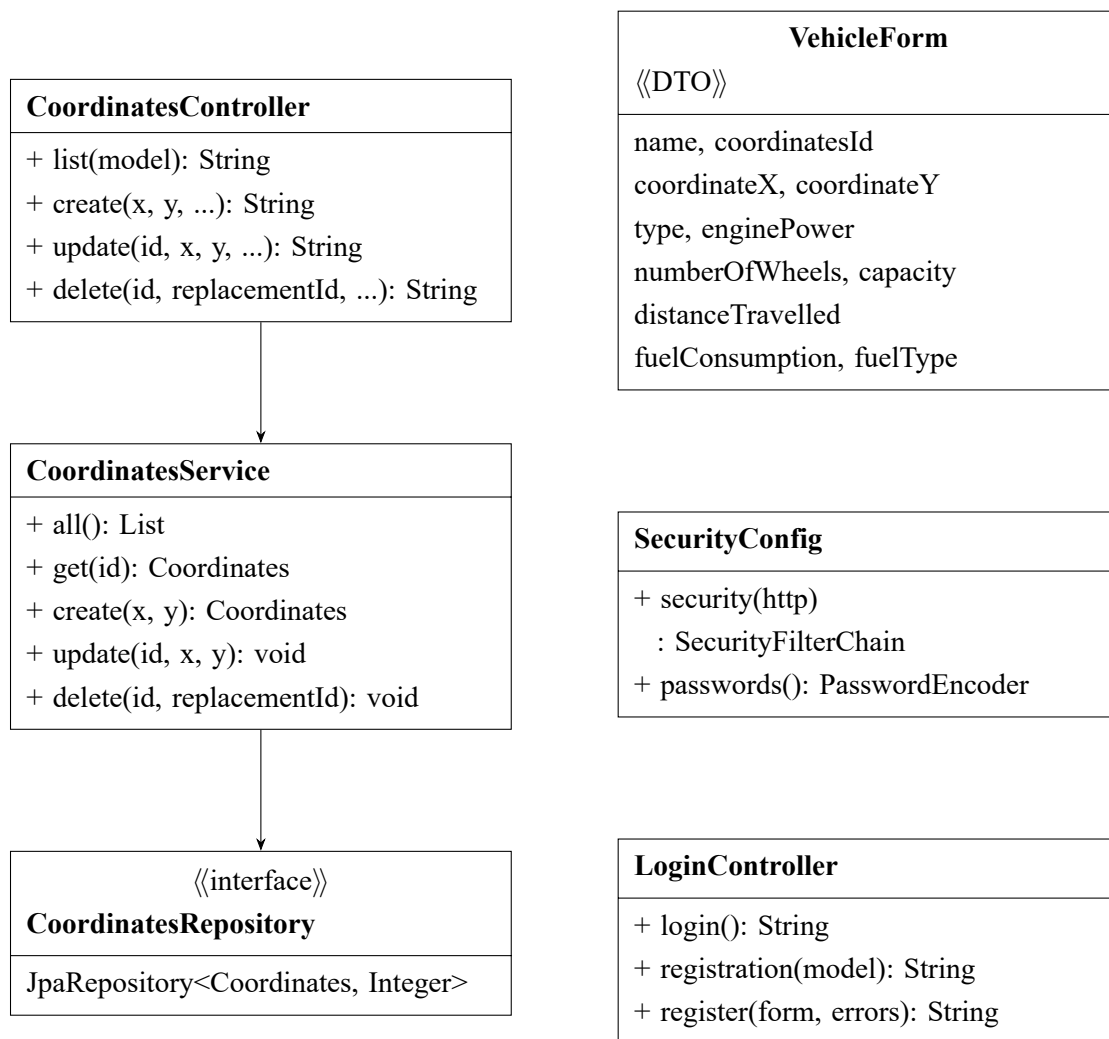


Рисунок 3 - Координаты, DTO и конфигурация входа

CoordinatesService также использует VehicleRepository для поиска и переназначения связанных объектов. VehicleService обращается к CoordinatesService при выборе либо создании координат. SecurityConfig задаёт обязательный вход; LoginController обрабатывает вход и регистрацию.

Регистрация и учётные записи

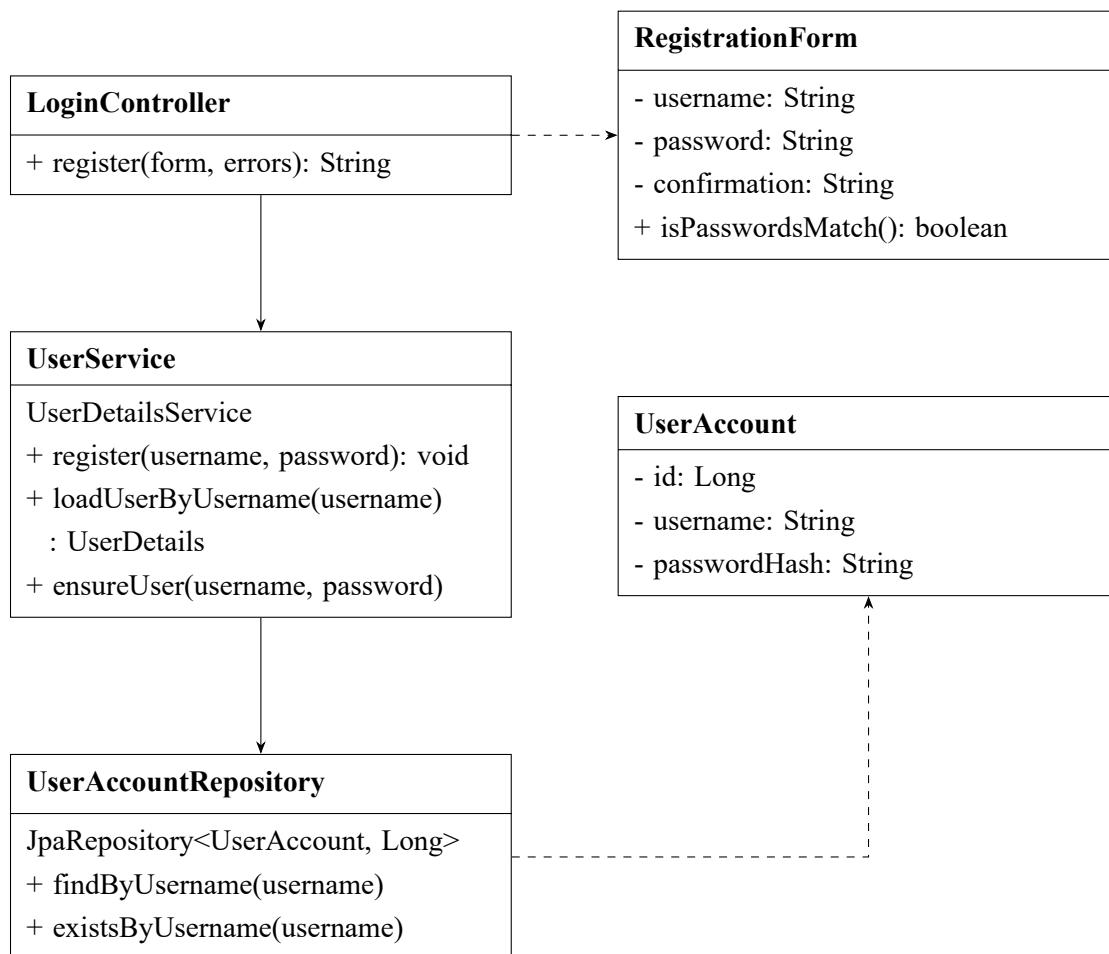


Рисунок 4 - Регистрация и хранение пользователей

Учётные записи хранятся в PostgreSQL. Логин уникален и приводится к нижнему регистру. Пароли хешируются BCrypt. `StartupData` создаёт начальные учётные записи и 24 примера транспорта; `SeedState` отмечает выполненное заполнение, исключая повторное добавление.

Обновление клиентов

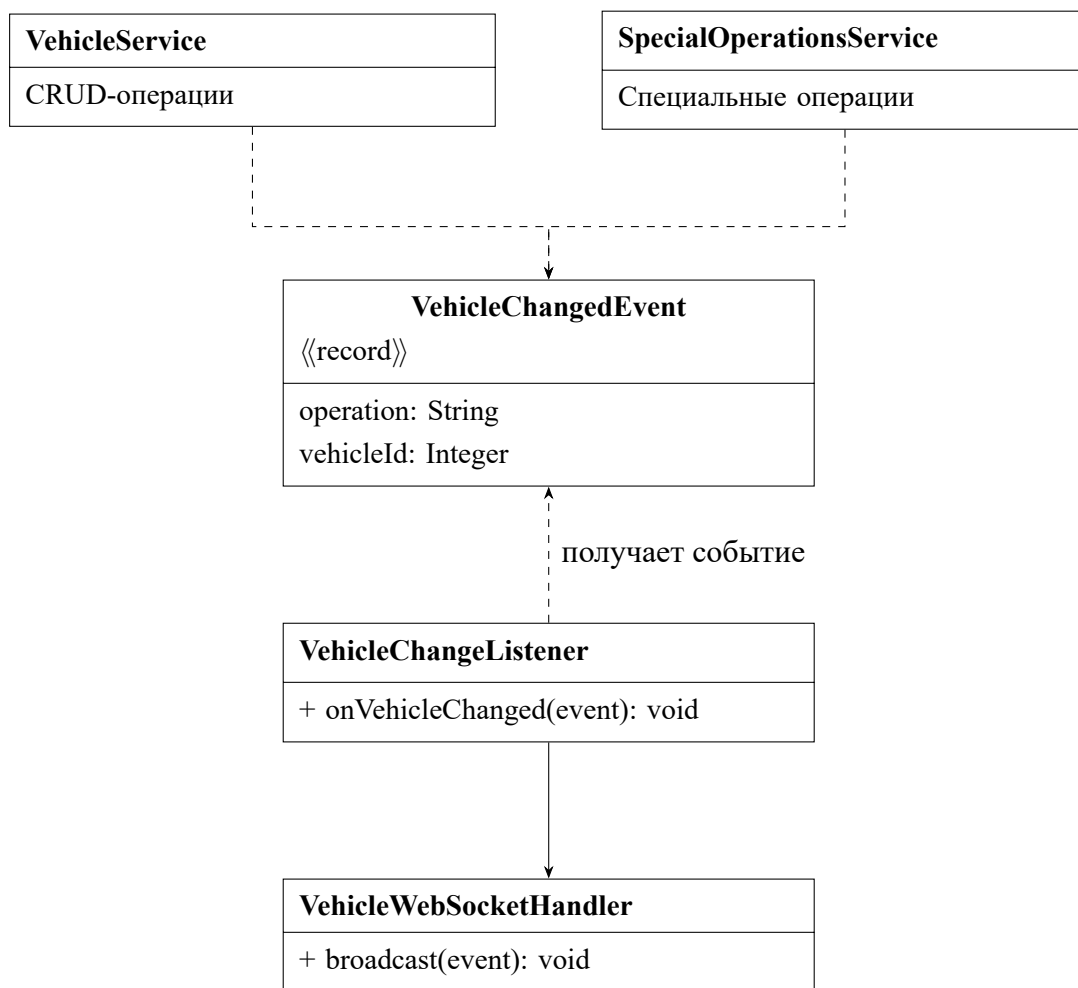


Рисунок 5 - Классы синхронизации

Сервисы, включая `CoordinatesService`, публикуют событие через механизм событий Spring. Listener обрабатывает его после фиксации транзакции. `WebSocketConfig` регистрирует обработчик по адресу `/ws/vehicles`. Пунктир обозначает зависимость.

Пакеты приложения

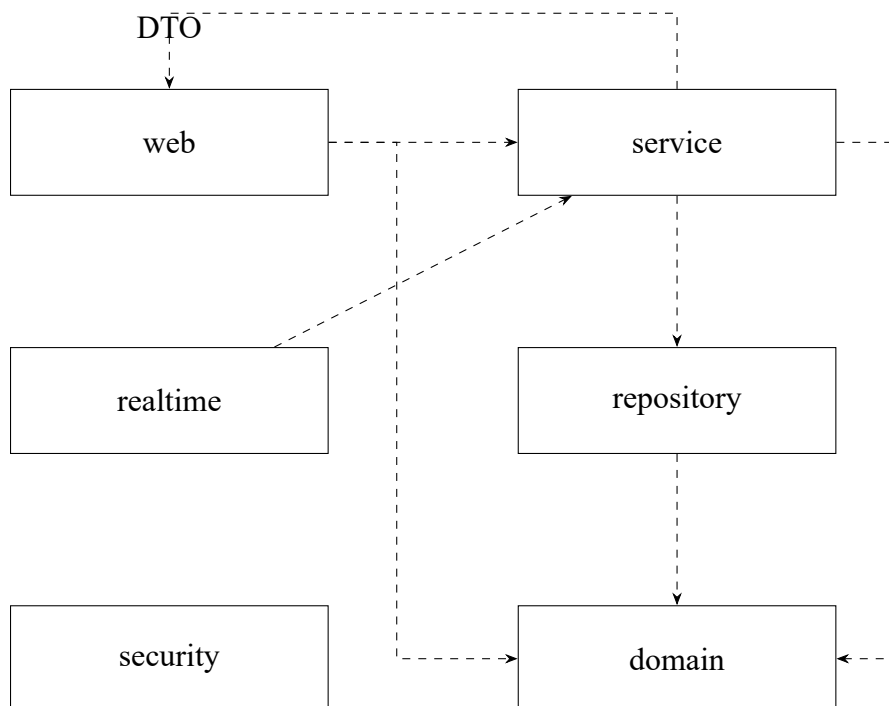


Рисунок 6 - Зависимости пакетов ru.itmo.vehiclelab

Пакет web содержит MVC-контроллеры, VehicleForm и обработчик ошибок; service - бизнес-логику, событие и исключение; repository - доступ к данным; domain - модель; realtime - WebSocket-компоненты; security - конфигурацию входа и защиты запросов.

Зависимость service от web обусловлена использованием VehicleForm. Пакет realtime использует VehicleChangedEvent из service. Hibernate выполняет отображение модели в PostgreSQL; HTML-шаблоны и статические ресурсы находятся отдельно от Java-пакетов.

3 Исходный код

Исходный код и конфигурация запуска опубликованы в репозитории:

`https://github.com/wonkersone/informatsionnye-sistemy-lr1`

Приложение использует Java 21, Spring MVC, Hibernate, Spring Data JPA, Thymeleaf и Flyway. Основной профиль подключается к PostgreSQL. Профиль demo использует временную H2-базу и предназначен только для демонстрации. Адрес локального интерфейса: `http://localhost:6767`.

Вход реализован средствами Spring Security; формы защищены CSRF-токенами. Координаты выбираются из существующих наборов либо создаются вместе с транспортом. При удалении используемых координат связанные транспортные средства переназначаются на выбранный набор в одной транзакции. Редактирование транспорта открывается в отдельном диалоговом окне.

4 Выводы

Разработана информационная система на Spring MVC и Hibernate с CRUD, пятью специальными операциями, регистрацией, авторизацией и синхронизацией клиентов. Реализованы хранение в PostgreSQL, проверка ограничений и переназначение связей.